chiselWare

PCS
Product User Guide

https://github.com/eddie-an/ClockDomainCrew-PCS

March 23, 2026

Contents

1 Errata and Known Issues **3**
1.1 Errata . . . 3
1.2 Known Issues . . . 3

2 Port Descriptions **4**

3 Parameter Descriptions **6**
3.1 Example Instantiation . . . 6

4 Theory of Operation **7**
4.1 Overview . . . 7
4.2 Architecture . . . 7
4.3 Transmit Path Logic . . . 7
4.3.1 64b/66b Encoding . . . 8
4.3.2 Scrambling . . . 8
4.3.3 Gearbox Handshaking . . . 8
4.4 Receive Path Logic . . . 8
4.4.1 Block Synchronization and Bitslip . . . 9
4.4.2 Descrambling and Decoding . . . 9
4.5 Link Monitoring and Diagnostics . . . 9
4.5.1 Bit Error Rate (BER) Monitor . . . 9
4.5.2 Watchdog Timer . . . 9
4.5.3 PRBS31 Support . . . 9
4.6 System Integration . . . 10

5 Simulation **11**
5.1 Tests . . . 11
5.2 Code coverage . . . 11
5.3 Running simulation . . . 11

6 Synthesis **12**
6.1 Area . . . 12
6.2 SDC File . . . 12
6.3 Timing . . . 12
6.4 Multicycle Paths . . . 12

1 Errata and Known Issues

This section tracks the current limitations and known deviations from the IEEE 802.3 10GBASE-R specification for the **Pcs** core.

1.1 Errata

There are currently no known functional bugs (errata) that deviate from the intended design of the 64b/66b PCS architecture.

1.2 Known Issues

The following items are known limitations or architectural constraints in the current release of the **Pcs** core:

- **Fixed Sync Header Width (hdrW):** While *hdrW* is presented as a parameter in the *PcsParams* class, there is **no functional degree of freedom** for this value. The 10GBASE-R standard and the internal protocol state machines (Encoder, Decoder, Frame Sync, and BER Monitor) are hard-coded for a 2-bit header. Attempting to elaborate the core with $hdrW \neq 2$ will result in a fatal Chisel elaboration-time error.
- **Self-Synchronizing Scrambler Latency:** The descrambler in the *PcsRxInterface* is a self-synchronizing additive scrambler using the $1+x^{39}+x^{58}$ polynomial. Upon achieving *rxBlockLock*, the output XGMII data will contain transient corruption for the first 58 bits (approximately one 64-bit clock cycle) until the internal LFSR state matches the transmitter.
- **PRBS31 Alignment Restriction:** The hardware bitslip (*serdesRxBitslip*) and reset (*serdesRxResetReq*) logic is explicitly suppressed when *cfgRxPrbs31Enable* is active. Link alignment must be established in operational mode before transitioning the core to PRBS31 test mode.
- **Gearbox Sensitivity:** The core does not implement an internal elastic buffer for rate matching between the 64-bit XGMII and 66-bit encoded blocks. The upstream MAC must strictly respect the *txGbxReqStall* signal to prevent data loss or frame corruption.

2 Port Descriptions

The ports for the **Pcs** core are shown below in Table 1. The width of several ports is controlled by the following parameters:

- *dataW* is the width of the internal data bus (typically 64 for 10GBASE-R)
- *ctrlW* is the width of the XGMII control bus, equal to *dataW*/8
- *hdrW* is the width of the 64b/66b sync header (fixed at 2 bits)

Port Name	Width	Direction	Description
Clock and Reset Signals			
rxClk	1	Input	Receive path clock (from SERDES/PMA)
rxRst	1	Input	Receive path active-high synchronous reset
txClk	1	Input	Transmit path clock (User/MAC domain)
txRst	1	Input	Transmit path active-high synchronous reset
XGMII Interface (MAC Side)			
xgmiiTxd	*dataW*	Input	TX data from MAC
xgmiiTxc	*ctrlW*	Input	TX control characters from MAC
xgmiiTxValid	1	Input	TX data valid from MAC
xgmiiRxd	*dataW*	Output	RX data to MAC
xgmiiRxc	*ctrlW*	Output	RX control characters to MAC
xgmiiRxValid	1	Output	RX data valid to MAC
MAC-Side Gearbox Handshaking			
txGbxReqSync	1	Output	Sync request to MAC
txGbxReqStall	1	Output	Stall/Backpressure request to MAC
txGbxSync	1	Input	Sync acknowledgment from MAC
SERDES Transmit Interface			
serdesTxData	*dataW*	Output	Scrambled data to SERDES
serdesTxDataValid	1	Output	Data valid to SERDES
serdesTxHdr	*hdrW*	Output	64b/66b sync header (01/10) to SERDES
serdesTxHdrValid	1	Output	Header valid to SERDES
serdesTxGbxReqSync	1	Input	Sync request from SERDES Gearbox
serdesTxGbxReqStall	1	Input	Stall request from SERDES Gearbox
serdesTxGbxSync	1	Output	Sync acknowledgment to SERDES
SERDES Receive Interface			
serdesRxData	*dataW*	Input	Raw data from SERDES
serdesRxDataValid	1	Input	Data valid from SERDES
serdesRxHdr	*hdrW*	Input	Raw sync header from SERDES
serdesRxHdrValid	1	Input	Header valid from SERDES

Port Name	Width	Direction	Description
serdesRxBitslip	1	Output	Pulse to SERDES to shift bit alignment
serdesRxResetReq	1	Output	Request to reset SERDES alignment logic
Status and Monitoring			
txBadBlock	1	Output	Invalid XGMII sequence detected during encoding
rxErrorCount	7	Output	PRBS error counter (if enabled)
rxBadBlock	1	Output	Invalid 64b/66b block detected during decoding
rxSequenceError	1	Output	XGMII protocol sequence violation detected
rxBlockLock	1	Output	64b/66b block alignment achieved
rxHighBer	1	Output	BER threshold exceeded ($> 10^{-4}$)
rxStatus	1	Output	Link UP status (Block Lock and low BER)
Test Configuration			
cfgTxPrbs31Enable	1	Input	Enable PRBS31 pattern generation on TX
cfgRxPrbs31Enable	1	Input	Enable PRBS31 pattern checking on RX

Table 1: PCS Port Descriptions

3 Parameter Descriptions

The hardware parameters for the **Pcs** core are described in Table 2. These parameters control the 64b/66b encoding, scrambling, and synchronization state machines required for 10GBASE-R operation.

Name	**Type**	**Min**	**Max**	**Description**
dataW	Int	32	64	Internal data bus width (32 or 64 bits)
ctrlW	Int	4	8	XGMII control bus width (dataW / 8)
hdrW	Int	2	2	64b/66b sync header width
txGbxIfEn	Boolean	false	true	Enable gearbox handshaking for TX path
rxGbxIfEn	Boolean	false	true	Enable gearbox handshaking for RX path
bitReverse	Boolean	false	true	Reverse bit order for SERDES compatibility
scramblerDisable	Boolean	false	true	Disable $x^{58} + x^{39} + 1$ scrambling logic
prbs31En	Boolean	false	true	Enable hardware PRBS31 pattern logic
txSerdesPipeline	Int	0	–	Pipeline stages for TX SERDES output
rxSerdesPipeline	Int	0	–	Pipeline stages for RX SERDES input
bitslipHighCycles	Int	0	–	Cycles to hold *serdesRxBitslip* high
bitslipLowCycles	Int	0	–	Cooldown cycles between bitslip pulses
count125Us	Double	–	–	Cycles representing a 125μs window for Bit Error Rate monitoring

Table 2: PCS Parameter Descriptions

3.1 Example Instantiation

The **Pcs** core is typically instantiated between the MAC and the high-speed Transceivers (SERDES). An example configuration using a 64-bit data path is shown below:

```
// Define PCS configuration
val myPcsParams = PcsParams(
  dataW = 64,
  ctrlW = 8,
  bitReverse = true,
  prbs31En = false
)

// Instantiate the PCS core
val ethernetPcs = Module(new Pcs(myPcsParams))

// Connect clocks (TX and RX are usually asynchronous)
ethernetPcs.io.txClk := macClock
ethernetPcs.io.rxClk := serdesRxClock
```

4 Theory of Operation

4.1 Overview

The **Pcs** core implements the Physical Coding Sublayer (PCS) for 10GBASE-R Ethernet as defined in IEEE 802.3 Clause 49. It serves as the bridge between the Media Access Control (MAC) layer's XGMII interface and the Physical Medium Attachment (PMA) / SERDES interface.

The core performs 64b/66b line coding, which includes data encapsulation, scrambling for DC balance, and block synchronization. The architecture is fully duplex, featuring independent Transmit (TX) and Receive (RX) paths. The PCS performs the following functions:

- 64b/66b block encoding and decoding
- Scrambling and descrambling for DC balance
- Block synchronization and alignment (Block Lock)
- High Bit Error Rate (BER) monitoring
- XGMII control character mapping
- Hardware PRBS31 test pattern support

4.2 Architecture

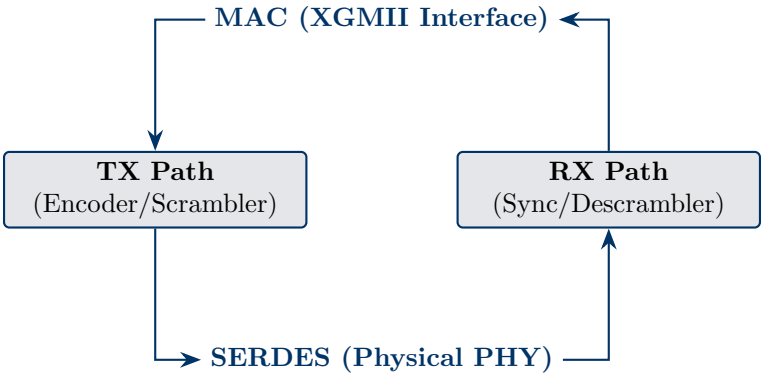

Figure 1: PCS Top-Level Architecture

4.3 Transmit Path Logic

The TX datapath converts 64-bit XGMII data and 8-bit control characters into a scrambled 66-bit bitstream.

Transmit Pipeline:

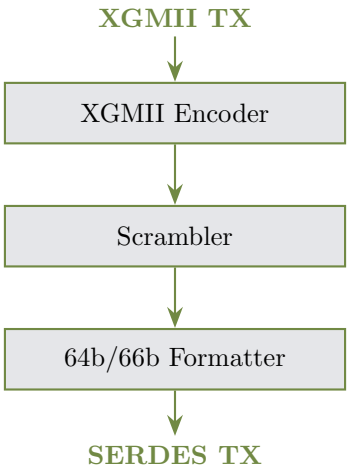

Figure 2: Transmit Datapath Pipeline

4.3.1 64b/66b Encoding

The *XgmiiEncoder* compresses XGMII characters into one of 15 valid 64b/66b block types. Each block is prepended with a 2-bit synchronization header:

- `b10`**:** Data Block (Contains only data payload).
- `b01`**:** Control Block (Contains start/terminate characters, ordered sets, or idles).

If the encoder receives an illegal XGMII sequence, it asserts *txBadBlock* and substitutes the data with an Error block (`0xFE`).

4.3.2 Scrambling

To ensure sufficient transition density and DC balance for high-speed serial clock recovery, the payload is processed by a self-synchronizing scrambler. It utilizes the polynomial $G(x) = 1 + x^{39} + x^{58}$. Similar to the MAC's CRC implementation, the **Pcs** core calculates the scrambler matrix at elaboration time to perform 64-bit parallel scrambling in a single clock cycle. The scrambler may be disabled through configuration.

4.3.3 Gearbox Handshaking

Because the 64b/66b encoding process increases the data rate by a factor of 66/64, the core implements gearbox handshaking. The *txGbxReqStall* signal provides backpressure to the MAC, ensuring that the TX path maintains the correct timing for the extra header bits without overflowing internal buffers.

4.4 Receive Path Logic

The Receive path recovers the 64b/66b block boundaries from the raw SERDES input and reverses the encoding process.

Receive Pipeline:

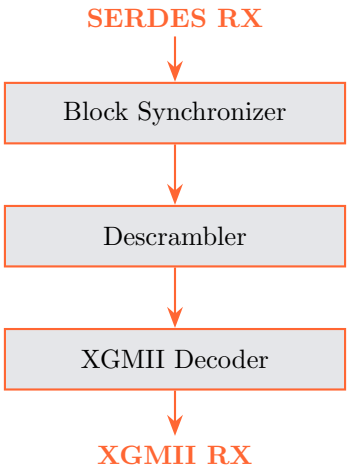

Figure 3: Receive Datapath Pipeline

4.4.1 Block Synchronization and Bitslip

Upon power-up or link loss, the *PcsRxFrameSync* module enters a search state. It monitors the 2-bit sync headers for valid patterns (`01` or `10`). If invalid headers are detected, the core asserts *serdesRxBitslip*, instructing the SERDES to shift the bit alignment. *rxBlockLock* is asserted once 64 consecutive valid headers are identified.

4.4.2 Descrambling and Decoding

Once block lock is achieved, the *PcsRxInterface* utilizes a self-synchronizing descrambler to recover the original payload. The *XgmiiDecoder* then expands the 64b/66b blocks back into standard XGMII data and control characters for the MAC.

4.5 Link Monitoring and Diagnostics

The PCS continuously monitors the bitstream for protocol violations. The primary error indicators are summarized in Table 3.

4.5.1 Bit Error Rate (BER) Monitor

The *PcsRxBerMon* module continuously validates sync headers within a moving 125μs window. If more than 16 errors are detected in this window, *rxHighBer* is asserted to notify the system of a degraded physical link.

4.5.2 Watchdog Timer

The *PcsRxWatchdog* acts as a protocol-level supervisor. It monitors for sustained "Bad Block" or "Sequence Error" counts. If the decoder reports excessive errors despite having block lock, the watchdog can trigger a *serdesRxResetReq* to force a full link re-alignment.

4.5.3 PRBS31 Support

For hardware validation, the core includes a Pseudo-Random Binary Sequence (PRBS31) generator and checker. When *cfgTxPrbs31Enable* is high, the TX path transmits a $2^{31} - 1$ pattern. The RX path can independently check this pattern and report bit errors via the 7-bit *rxErrorCount* register.

Signal	Description
`rxBadBlock`	Asserted when an invalid 64b/66b block type is detected.
`rxSequenceError`	Indicates a violation in the XGMII control character sequence.
`rxHighBer`	Asserted when the Bit Error Rate exceeds the 10^{-4} threshold.

Table 3: PCS Error Indicators

4.6 System Integration

To integrate the **Pcs** core into a larger system, the following interfaces must be connected:

1. **MAC Interface:** Connects to the MAC core via the 64-bit XGMII interface.
2. **SERDES Interface:** Connects to high-speed FPGA transceivers (GTH/GTY) via a 64-bit (or 32-bit) raw data interface.
3. **Handshaking:** Uses gearbox stall signals to regulate data flow, ensuring that the additional 2 bits of 64b/66b overhead do not cause buffer overflows in the upstream MAC.

Figure 4 illustrates how the **Pcs** core fits within a complete 10GBASE-R architecture, showing the flow of data from the application layer down to the physical medium and back up.

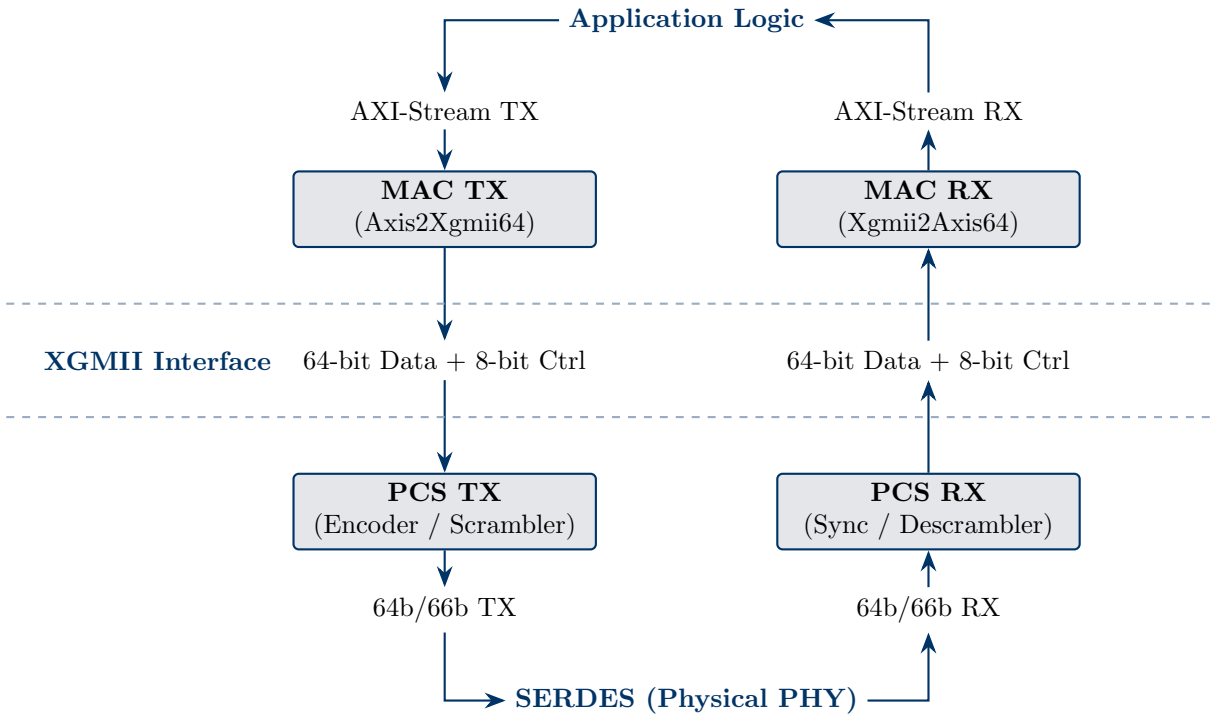

Figure 4: Combined MAC and PCS 10GBASE-R Architecture

5 Simulation

5.1 Tests

The included test bench tests the default configuration of the Pcs. More configurations can be added as desired.

5.2 Code coverage

The tests exceed the chiselWare minimum standard of 90% Statement coverage and 95% Branch coverage.

5.3 Running simulation

Simulations can be run directly from the command prompt as follows:

```
$ sbt "project core" test
```

or from make as follows:

```
$ make test
```

6 Synthesis

6.1 Area

The **Pcs** has been tested in a number of configurations and the following results should be representative of what a user should see in their own technology.

Config Name	width	Gates
pcs_default[1]	64-bit	6,686

Table 4: PCS Synthesis Results

6.2 SDC File

An `.sdc` file is generated to provide synthesis and static timing analysis tools guidance for synthesis.

The `Pcs.sdc` file can be found in the test case directory as follows:
`./modules/pcs/generated/synTestCases/<test case name>/Pcs.sdc`

6.3 Timing

The following timing was extracted using the generated .sdc files with the open-source Nangate 45nm library.

Config Name	Period	Duty Cycle	Input Delay	Output Delay	Slack
pcs_default[1]	6.4ns	50%	1.28ns	1.28ns	+4.40ns (MET)

Table 5: Static Timing Analysis results

[1] The default configuration is defined in the case class of `PcsParams.scala`.

6.4 Multicycle Paths

None. The design is architected to complete all encoding, scrambling, and residue checks within a single clock cycle at 156.25 MHz.